

Circular References in Python Programming

**CWC**

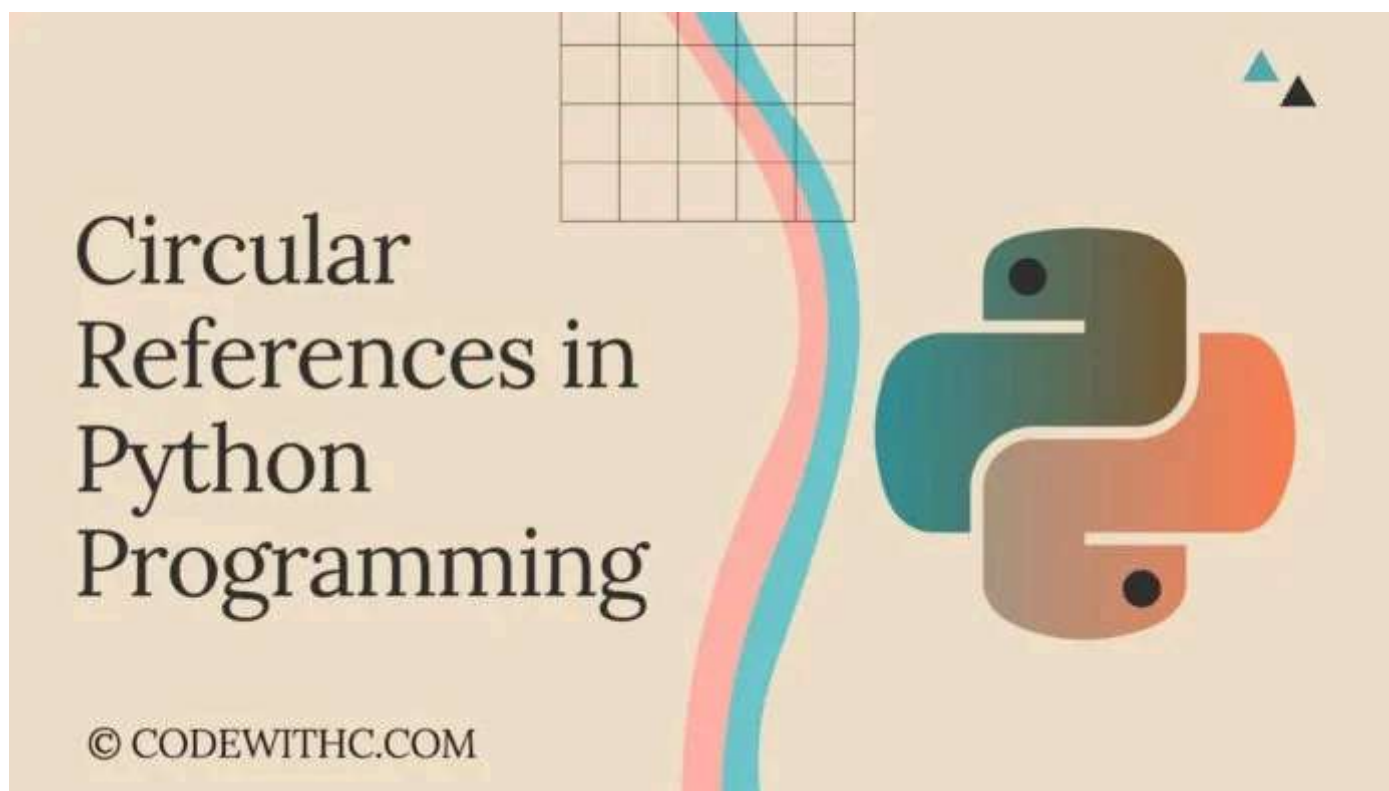
Last updated: July 5, 2022 9:13 am

[Share](#) [f](#) [X](#) [WhatsApp](#) 5 Min Read

Circular References in Python Programming

Circular references in Python a programming language are caused by variables referring to themselves (or to other variables). In some languages, the cause of the problem can be eliminated by declaring the variable explicitly as a reference rather than a copy of itself. Python does not have this feature, so it is necessary to use other techniques to eliminate circular references in a Python program.

A python program has no defined structure. For this reason, it is common practice to put one function at the top level of the program, and call it at the end of every program. This way, there is a single point of entry and exit to the program. If there are other functions, then these should be nested within the main function, which acts as the parent.



—— Circular References in Python Programming

The concept of a circular reference is illustrated in the following diagram. Here, the object 'B' refers to itself in the field 'A'.

This code produces an error because A refers to B which refers to A. The error is called a 'circular reference' because the same object is referred to more than once. Circular references are the result of the programmer using the wrong data structures and the problem must be solved manually.

The following code uses a set to avoid this type of problem.

```
>>> a=set()
>>> a.add(a)
>>> print(a)
{}
>>> a.add(a)
Traceback (most recent call last):
  File "", line 1, in
AttributeError:'set' object has no attribute 'add'
```

[COPY CODE](#)

If the set is modified, then the list containing the set will be updated automatically. The following program illustrates this idea.

```
>>> a=set()
>>> b=set()
>>> a.update(b)
>>> print(a)
{}
>>> a.update(a)
>>> print(a)
{1}
```

[COPY CODE](#)

The following program demonstrates the use of another data structure, the dictionary, to avoid circular references.

```
>>> a={}
>>> b=a
```

[COPY CODE](#)

```
>>> a[b]=1
>>> print(a)
{}
>>> print(b)
{1}
```

Python programmers tend to use these methods to avoid circular references, but they do not always work. Some programs require circular references.

A circular reference occurs when two or more objects refer to each other. The problem is that if A refers to B, then B refers back to A. Therefore, a chain of references occurs.

[COPY CODE](#)

```
>>> a=5
>>> b=a
>>> c=b
>>> d=c
>>> print(a)
5
>>> print(b)
5
>>> print(c)
5
>>> print(d)
5
```

In this program, a, b, and c all refer to each other and d refers back to c, which refers back to d, which refers back to c, which refers back to a. So the references form a loop.

[COPY CODE](#)

```
>>> a=[a,b,c,d]
>>> b=5
>>> print(a)
[5, 5, 5, 5]
```

The following program shows how a set will not allow a circular reference to occur.

[COPY CODE](#)

```
>>> a=set()
>>> b=a
>>> a.add(b)
>>> print(a)
{}
>>> a.add(a)
>>> print(a)
{1}
```

[COPY CODE](#)

```
import time
def create_pair():
    a = {'name' : 'Foo'}
    b = {'name' : 'Bar'}
    a['pair'] = b
```

```
b['pair'] = a
#print(a)
for i in range(1, 30000000):
    create_pair()
print("let's sleep now a bit")
time.sleep(20)
```

 **Share This Article**

 Facebook

 Twitter

[◀ PREVIOUS ARTICLE](#)

[NEXT ARTICLE ▶](#)



How to Make C Programming Program Easier?

How to Find Out Which Class an Object (Instance) Belongs to in Python Programming



Leave a comment

Your email address will not be published. Required fields are marked *

Leave a comment

Your name

Your email

Your Website

Post Comment

Follow US |   

© 2024 CodeWithC.com. All Rights Reserved.



[Privacy & Cookies Policy](#)

 English

 English